

DevOps Shack | devopsshack.com | @devopsshack

50 Linux ASSIGNMENTS

with Solutions, Explanations & Pro Tips

50

Assignments

11

Categories

3

Difficulty Levels

200+

Commands Covered

CATEGORY	ASSIGNMENTS	DIFFICULTY
File & Directory Management	5 assignments	Intermediate, Advanced
Text Processing	5 assignments	Advanced, Intermediate
User & Permissions	5 assignments	Advanced, Intermediate
Process Management	5 assignments	Intermediate, Advanced
Networking	5 assignments	Advanced
Disk & Storage	5 assignments	Advanced, Intermediate
System Administration	5 assignments	Advanced, Intermediate
Shell Scripting	5 assignments	Advanced
Security & Hardening	5 assignments	Advanced, Intermediate
Package Management	4 assignments	Advanced, Intermediate
Performance Tuning	1 assignments	Advanced

Aditya Jaiswal | @devopsshack | devopsshack.com

TABLE OF CONTENTS

File & Directory Management	4
01. Recursive File Search with Multiple Conditions	4
02. Watch a Directory for Changes in Real-Time	4
03. Hard Links vs Symbolic Links Deep Dive.....	5
04. Secure File Deletion and Recovery Prevention	6
05. Bulk File Organization by Date.....	7
Text Processing	8
06. Multi-Pattern Log Analysis with AWK.....	8
07. Advanced sed for Config Transformation.....	8
08. JSON Processing with jq.....	9
09. Diff and Patch Workflow	10
10. Real-time Log Filtering Pipeline	11
User & Permissions	13
11. Fine-Grained Access with ACLs	13
12. Sudo Privilege Escalation Audit	13
13. Password Aging and Account Expiry	15
14. PAM Configuration for Security	16
15. umask and Default Permissions Strategy	17
Process Management.....	18
16. Process Tree Visualization and Analysis	18
17. Systemd Service Creation from Scratch	19
18. cgroups Resource Throttling	20
19. Signal Handling in Shell Scripts	21
20. CPU Affinity and Process Pinning.....	22
Networking	24
21. Advanced iptables NAT and Port Forwarding	24
22. Network Namespace Isolation	25
23. SSL/TLS Certificate Management.....	26
24. Network Bonding and Teaming.....	27
25. Advanced SSH Tunneling and Port Forwarding	28
Disk & Storage	29
26. LVM (Logical Volume Manager) Setup and Expansion	29
27. RAID Configuration with mdadm	30
28. ZFS Filesystem Operations.....	31
29. Disk Benchmarking and I/O Analysis	32
30. Automated Incremental Backup with rsync.....	33
System Administration	35
31. Kernel Module Management	35
32. System Resource Monitoring Dashboard	36

33. Audit Framework with auditd	37
34. Systemd Journal Advanced Management	38
35. Boot Process Optimization and Troubleshooting	39
Shell Scripting	41
36. Parallel Job Execution Framework.....	41
37. Configuration Template Engine.....	42
38. Database Backup with Rotation and Encryption	43
39. Infrastructure Drift Detection	45
40. Multi-Environment Deployment Script.....	46
Security & Hardening	48
41. Comprehensive System Hardening Script	48
42. Intrusion Detection with Tripwire/AIDE.....	49
43. Fail2ban Custom Jail Configuration	50
44. GPG Key Management and File Signing	51
45. AppArmor Profile Creation	53
Package Management	55
46. Custom APT/YUM Repository Setup	55
47. Snap and Flatpak Management	56
48. Dependency Resolution and Conflict Management.....	57
49. Container Image Layer Analysis	58
Performance Tuning	60
50. Complete Production Server Tuning	60

01

File & Directory Management

01. Recursive File Search with Multiple Conditions

FILE & DIRECTORY MANAGEMENT

INTERMEDIATE

#1

TASK

Find all .log files in /var that are larger than 50MB, older than 7 days, and not accessed in the last 24 hours. Display their full path and size.

SOLUTION

```
find /var -type f -name '*.log' -size +50M -mtime +7 -atime +1 -exec du -sh {} \;
2>/dev/null
```

EXPLANATION

find supports multiple conditions applied simultaneously (AND by default). -type f restricts to regular files. -name '*.log' matches log files. -size +50M finds files >50MB. -mtime +7 means modified more than 7 days ago. -atime +1 means not accessed in the last 24 hours. -exec du -sh {} \; runs disk usage on each match. 2>/dev/null silences permission errors on restricted directories.

PRO TIP

Use -ls instead of -exec ls -la {} \; for faster execution — it's a built-in find action.

REAL WORLD

Essential for log management automation scripts in production servers with limited disk space.

02. Watch a Directory for Changes in Real-Time

FILE & DIRECTORY MANAGEMENT

ADVANCED

#2

TASK

Monitor /etc/nginx/conf.d/ for any file creation, modification, or deletion events, and log changes with timestamps.

SOLUTION

```
inotifywait -m -r --format '%T %e %w%f' --timefmt '%Y-%m-%d %H:%M:%S' -e
create,modify,delete /etc/nginx/conf.d/ | tee /var/log/nginx_config_changes.log
```

EXPLANATION

inotifywait (from inotify-tools) uses the Linux kernel's inotify subsystem for efficient file system event monitoring. -m runs in monitor mode indefinitely. -r watches recursively. --format controls output format: %T=timestamp, %e=event type, %w%f=full path. --timefmt defines the timestamp format. -e specifies which events to watch. tee writes to both stdout and log file simultaneously.

PRO TIP

In CI/CD pipelines, use inotifywait to trigger config validation and reload when deployment updates config files.

REAL WORLD

Used in GitOps workflows to auto-apply changes when config files are updated by deployment tools.

03. Hard Links vs Symbolic Links Deep Dive

FILE & DIRECTORY MANAGEMENT**INTERMEDIATE**

#3

TASK

Create a hard link and a soft link to /opt/app/config.yaml. Demonstrate the difference using inode numbers. Then delete the original and test both links.

SOLUTION

```
# Check original inode
ls -li /opt/app/config.yaml

# Create hard link
ln /opt/app/config.yaml /opt/app/config.hard.yaml

# Create soft link
ln -s /opt/app/config.yaml /opt/app/config.soft.yaml

# Compare inodes
ls -li /opt/app/config*.yaml

# Delete original
rm /opt/app/config.yaml

# Hard link still works (same inode, data intact)
cat /opt/app/config.hard.yaml

# Soft link is now broken (dangling)
cat /opt/app/config.soft.yaml # Error: No such file
```

EXPLANATION

A hard link creates another directory entry pointing to the same inode (same physical data on disk). Both the original and hard link share identical inode numbers — deleting either one leaves the data accessible via the other until ALL hard links are removed. A symbolic (soft) link stores the path to the target; if the

target is removed, the symlink becomes a 'dangling' pointer. Hard links cannot span filesystems or link to directories (to prevent loops). Use `ls -li` to see inode numbers — hard links will match the original.

PRO TIP

Use `find / -inum <inode_number>` to discover all hard links to a specific inode across the filesystem.

REAL WORLD

Package managers use hard links extensively for deduplication. Node.js's `node_modules/` benefits from hard-link-based tools like `pnpm`.

04. Secure File Deletion and Recovery Prevention

FILE & DIRECTORY MANAGEMENT**INTERMEDIATE**

#4

TASK

Securely delete a file containing sensitive data (`passwords.txt`) so it cannot be recovered with standard forensic tools.

SOLUTION

```
# Overwrite with random data 3 times, then delete
shred -v -n 3 -u passwords.txt

# Or using dd for more control:
dd if=/dev/urandom of=passwords.txt bs=1M count=1 conv=notrunc
rm passwords.txt

# For SSDs (shred is less effective due to wear leveling):
cryptsetup luksErase /dev/sdX # If using LUKS encryption
```

EXPLANATION

Regular `rm` only removes the directory entry, leaving data on disk until overwritten. `shred` overwrites the file content multiple times with random data before deletion. `-v` shows progress, `-n 3` specifies 3 overwrite passes, `-u` unlinks (deletes) the file after. Note: `shred` is less effective on SSDs, flash storage, and filesystems with journaling (ext4 journal may preserve data copies), CoW filesystems (btrfs, ZFS), or RAID arrays. For SSDs, use ATA Secure Erase or encryption-at-rest.

PRO TIP

The most reliable secure deletion on SSDs is to use full-disk encryption from day one — then 'deleting' the encryption key renders all data unreadable.

REAL WORLD

Required compliance practice for PCI-DSS, HIPAA, and SOC2 environments when decommissioning servers.

05. Bulk File Organization by Date

FILE & DIRECTORY MANAGEMENT

INTERMEDIATE

#5

TASK

Organize all .jpg files in /data/photos into subdirectories by year/month based on the file's modification date (e.g., /data/photos/2024/03/).

SOLUTION

```
find /data/photos -maxdepth 1 -name "*.jpg" -type f | while read -r file; do
    year=$(date -r "$file" +%Y)
    month=$(date -r "$file" +%m)
    dest="/data/photos/$year/$month"
    mkdir -p "$dest"
    mv "$file" "$dest/"
    echo "Moved: $file -> $dest/"
done
```

EXPLANATION

find with -maxdepth 1 processes only the top-level directory (non-recursive). The while read loop processes each filename safely including those with spaces. date -r reads the file's modification timestamp. +%Y and +%m extract the 4-digit year and 2-digit month respectively. mkdir -p creates the nested directory structure in one shot. This approach handles edge cases like already-existing destination directories gracefully.

PRO TIP

For EXIF-based organization (actual photo date, not file system date), use exiftool: `exiftool '-Directory<DateTimeOriginal' -d '/data/photos/%Y/%m' *.jpg`

REAL WORLD

Standard pattern for media server automation, photography workflows, and compliance archive organization.

02

Text Processing

Assignments 1, Assignments 6-10

06. Multi-Pattern Log Analysis with AWK

TEXT PROCESSING

ADVANCED

#6

TASK

Parse `/var/log/nginx/access.log` to generate a report showing: total requests, unique IPs, top 5 URLs, and average response size.

SOLUTION

```
awk '{
    total++
    ips[$1]++
    urls[$7]++
    size += $10
}
END {
    print "Total Requests:", total
    print "Unique IPs:", length(ips)
    print "Avg Response Size:", int(size/total), "bytes"
    print "\nTop 5 URLs:"
    for (u in urls) print urls[u], u | "sort -rn | head -5"
}' /var/log/nginx/access.log
```

EXPLANATION

AWK processes text line by line. The main block runs for every line: `total++` counts lines. `ips[$1]++` builds an associative array keyed by IP (field 1 in combined log format). `urls[$7]++` tracks URL frequency (field 7). `size += $10` accumulates response bytes (field 10). The END block runs after all lines. `length(ips)` counts unique keys. Piping to `'sort -rn | head -5'` inside AWK sorts the top URLs. The combined log format fields are: IP, ident, auth, time, request, status, bytes, referer, user-agent.

PRO TIP

Use `GoAccess` for real-time interactive `nginx/apache` log analysis with the same log files:
`goaccess /var/log/nginx/access.log -o report.html --log-format=COMBINED`

REAL WORLD

Daily log analysis is a core DevOps responsibility for capacity planning, anomaly detection, and SLA reporting.

07. Advanced sed for Config Transformation

TEXT PROCESSING

ADVANCED

#7

TASK

Transform a Docker Compose file to replace all 'latest' image tags with a specific version '2.4.1', uncomment all lines starting with '#PORT', and add a network section at the end.

SOLUTION

```
sed -i \
  -e 's/:latest/:2.4.1/g' \
  -e 's/^#PORT/PORT/' \
  -e '/^$/{ /networks/!a\nnetworks:\n  app_net:\n    driver: bridge
}' \
  docker-compose.yml

# Cleaner approach for appending:
sed -i 's/:latest/:2.4.1/g; s/^#PORT/PORT/' docker-compose.yml
cat >> docker-compose.yml << 'EOF'

networks:
  app_net:
    driver: bridge
EOF
```

EXPLANATION

sed -i with -e allows multiple expressions in one pass. 's/:latest/:2.4.1/g' globally replaces :latest tags. '^#PORT' matches lines starting with #PORT and the substitution removes the # to uncomment. For appending structured content, using a heredoc (cat >>) is cleaner and more maintainable than complex sed append commands. Always test sed commands without -i first to verify output before modifying files in place.

PRO TIP

Use sed -i.bak to create automatic backups: the .bak suffix is appended to the original filename before modification.

REAL WORLD

CI/CD pipelines use these patterns to inject build numbers into deployment manifests before applying to Kubernetes.

08. JSON Processing with jq**TEXT PROCESSING****INTERMEDIATE**

#8

TASK

From a Kubernetes pod status JSON, extract: pod name, status, container restart counts, and any containers not in Running state.

SOLUTION

```
# Get all pods summary
kubectl get pods -o json | jq '.items[] | {
  name: .metadata.name,
  status: .status.phase,
  restarts: [.status.containerStatuses[]?.restartCount] | add,
  notReady: [.status.containerStatuses[]? | select(.state.running == null) | .name]
}'

# Filter only problematic pods
kubectl get pods -o json | jq '.items[] | select(.status.phase != "Running") | {
  name: .metadata.name,
  phase: .status.phase,
  reason: .status.reason
}'
```

EXPLANATION

jq is a lightweight JSON processor. `.items[]` iterates the pod array. Object construction `{}` creates new JSON objects. `.status.containerStatuses[]?` uses the `?` operator to safely handle null (no crash if missing). `[... | add]` sums restart counts across all containers. `select()` filters items matching a condition. `.state.running == null` identifies containers not in running state. This is vastly more reliable and maintainable than parsing kubectl output with `grep/awk`.

PRO TIP

Store jq queries in files: `jq -f query.jq input.json`. Combine with `watch`: `watch 'kubectl get pods -o json | jq ...'` for live monitoring.

REAL WORLD

Essential for Kubernetes operators, Helm chart debugging, and automated health-check scripts in GitOps pipelines.

09. Diff and Patch Workflow

TEXT PROCESSING

INTERMEDIATE

#9

TASK

Compare two versions of a config file, create a patch, apply it to a third server, and verify the result.

SOLUTION

```
# Compare and create patch
diff -u nginx.conf.orig nginx.conf.new > nginx.patch

# Review the patch before applying
cat nginx.patch

# Apply patch (dry run first)
patch --dry-run nginx.conf.orig < nginx.patch

# Apply for real
patch nginx.conf.orig < nginx.patch

# Verify
```

```
diff nginx.conf.orig nginx.conf.new # Should show no output
# Revert if needed
patch -R nginx.conf.orig < nginx.patch
```

EXPLANATION

diff -u generates a unified diff (context format with +/- lines). This format is standard and human-readable. The patch file contains the original filename, timestamp, context lines, and exact changes. patch applies the diff to the original file — it's smart enough to find the right location even if line numbers shifted slightly. --dry-run tests if the patch applies cleanly without modifying anything. -R reverses a patch (undoes it). Patch files can be stored in version control as configuration change records.

PRO TIP

Use git diff and git apply for patches in code repos — it handles binary files, permissions, and renames that diff/patch miss.

REAL WORLD

Patch workflows are used in kernel development, Ansible file modules, and config management for tracking changes across environments.

10. Real-time Log Filtering Pipeline

TEXT PROCESSING

ADVANCED

#10

TASK

Create a live monitoring pipeline that watches /var/log/syslog, filters for ERROR or CRITICAL messages, highlights them in color, and sends an alert when more than 5 errors occur per minute.

SOLUTION

```
tail -F /var/log/syslog | \
  grep --line-buffered -E 'ERROR|CRITICAL|FATAL' | \
  grep --line-buffered -v 'DEBUG' | \
  awk 'BEGIN{count=0; start=systemtime()} {
    # Colorize output
    gsub(/ERROR/, "\033[31mERROR\033[0m")
    gsub(/CRITICAL/, "\033[1;31mCRITICAL\033[0m")
    print
    count++
    if (systemtime() - start >= 60) {
      if (count > 5) print "\033[41m ALERT: " count " errors in last minute! \033[0m"
      count=0; start=systemtime()
    }
  }'
```

EXPLANATION

`tail -F` (capital F) follows the file and handles log rotation gracefully by reopening when the file is renamed/recreated. `--line-buffered` ensures `grep` passes each line immediately (not in blocks), critical for real-time pipelines. `AWK`'s `systime()` returns Unix timestamp for time-window calculations. ANSI escape codes (`\033[31m` = red, `\033[0m` = reset) add color. `gsub()` performs global regex substitution within `AWK`. The sliding window tracks error count and resets every 60 seconds.

PRO TIP

For production alerting, integrate with PagerDuty or Slack: `curl -X POST -H 'Content-type: application/json' --data '{"text":"Alert!"}' SLACK_WEBHOOK_URL`

REAL WORLD

Foundation for building lightweight monitoring agents before Prometheus/Grafana is set up in a new environment.

03

User & Permissions

5 assignments | Assignments 14 - 18

11. Fine-Grained Access with ACLs

USER & PERMISSIONS

ADVANCED

#11

TASK

Set up `/data/project` so that: owner has full access, group 'developers' has read/write, user 'qa_bot' has read-only, and 'intern_user' has no access despite being in 'developers'.

SOLUTION

```
# Enable ACL on filesystem (if not already)
mount -o remount,acl /data

# Set base permissions
chmod 770 /data/project

# Add ACL entries
setfacl -m u:qa_bot:r-x /data/project
setfacl -m u:intern_user:--- /data/project
setfacl -m g:developers:rwx /data/project

# Set default ACLs for new files
setfacl -d -m u:qa_bot:r-x /data/project
setfacl -d -m g:developers:rwx /data/project

# Verify
getfacl /data/project
```

EXPLANATION

POSIX Access Control Lists (ACLs) extend standard Unix permissions to allow per-user and per-group access rules beyond the owner/group/other model. `setfacl -m` sets (modifies) ACL entries. `u:` prefix targets a user, `g:` targets a group. Permission format is `rw` or `---` for no access. The `-d` flag sets default ACLs, which are inherited by new files and directories created inside. A user ACL (`intern_user:---`) overrides group membership — the most specific rule wins. `getfacl` displays all ACL entries including defaults.

PRO TIP

ACLs are silently dropped during some copy operations. Use `cp -a` or `rsync -a --acls` to preserve them. Check with: `ls -la` (a '+' after permissions means ACL is set).

REAL WORLD

ACLs are standard in enterprise NFS shares, CI/CD build directories with multiple pipeline users, and shared development servers.

12. Sudo Privilege Escalation Audit

USER & PERMISSIONS

ADVANCED

#12

TASK

Write a script that audits all sudo rules on the system, identifies users who can run commands as root, and flags dangerous permissions like NOPASSWD and ALL.

SOLUTION

```
#!/bin/bash
echo "=== SUDO PRIVILEGE AUDIT ==="
echo "Generated: $(date)"
echo ""

# Parse sudoers and included files
for file in /etc/sudoers $(ls /etc/sudoers.d/ 2>/dev/null | xargs -I{} echo /etc/sudoers.d/{}); do
    [ -f "$file" ] || continue
    echo "--- File: $file ---"
    grep -v "^#" "$file" | grep -v "^$" | while read -r line; do
        echo "    $line"
        echo "$line" | grep -q "NOPASSWD" && echo "    [!] WARNING: NOPASSWD detected"
        echo "$line" | grep -q "(ALL)" && echo "    [!] WARNING: ALL hosts allowed"
        echo "$line" | grep -qE "^[^%].*ALL=.*ALL" && echo "    [!] CRITICAL: Full root access"
    done
done
echo ""

echo "=== USERS IN SUDO/WHEEL GROUP ==="
grep -E "^sudo:|^wheel:" /etc/group | cut -d: -f4
```

EXPLANATION

The script iterates through /etc/sudoers and all files in /etc/sudoers.d/ (the include directory). grep -v removes comments and blank lines. Each non-comment line is checked for security red flags: NOPASSWD means the user never needs to enter a password (major risk), (ALL) in the runas position means the command can be run as any user, and the pattern matching full root access catches the most dangerous entries. The final section reads /etc/group to list group-based sudo access.

PRO TIP

Run `sudo -l -U username` to see effective sudo rules for any specific user including group-inherited permissions.

REAL WORLD

Required for SOC2, ISO 27001, and CIS Benchmark compliance audits. Automated with tools like Lynis or OpenSCAP.

13. Password Aging and Account Expiry

USER & PERMISSIONS

INTERMEDIATE

#13

TASK

Configure password policy: minimum 8-day change interval, maximum 90-day validity, 14-day warning before expiry. Apply to user 'webadmin' and verify.

SOLUTION

```
# Set password aging for webadmin
chage -m 8 -M 90 -W 14 webadmin

# Set account expiry date
chage -E 2025-12-31 webadmin

# Force password change on next login
chage -d 0 webadmin

# View all aging info
chage -l webadmin

# Set system-wide defaults in /etc/login.defs
grep -E 'PASS_MAX_DAYS|PASS_MIN_DAYS|PASS_WARN_AGE' /etc/login.defs

# These defaults apply only to NEW users:
# PASS_MAX_DAYS    90
# PASS_MIN_DAYS    8
# PASS_WARN_AGE    14
```

EXPLANATION

chage (change age) manages password and account aging in /etc/shadow. -m sets minimum days between changes (prevents immediately resetting back to old password). -M sets maximum validity days. -W sets warning days before expiry. -E sets account expiration date (not password expiry). -d 0 sets last password change to epoch (Jan 1, 1970), forcing immediate change on next login. /etc/login.defs sets global defaults but only affects newly created accounts — existing users must be updated individually with chage.

PRO TIP

Use `passwd --stdin` for scripted password setting: `echo 'newpass' | passwd --stdin username` (RHEL/CentOS). On Debian use `chpasswd`: `echo 'user:pass' | chpasswd`

REAL WORLD

Compliance requirements like PCI-DSS mandate maximum 90-day password rotation and account lockout policies.

14. PAM Configuration for Security

USER & PERMISSIONS

ADVANCED

#14

TASK

Configure PAM to: lock an account after 3 failed login attempts for 5 minutes, enforce password complexity (uppercase, number, special char), and log all authentication events.

SOLUTION

```
# /etc/pam.d/common-auth (Debian/Ubuntu) - add account lockout:
# auth required pam_tally2.so onerr=fail audit silent deny=3 unlock_time=300
# Or with pam_faillock (newer systems):
# auth required pam_faillock.so preauth audit silent deny=3 unlock_time=300
# /etc/pam.d/common-password - add complexity:
# password requisite pam_pwquality.so retry=3 minlen=12 ucredit=-1 dcredit=-1 ocredit=-1
# Check failed login count:
faillock --user webadmin
# Unlock manually:
faillock --user webadmin --reset
# Or with pam_tally2:
pam_tally2 --user=webadmin --reset
```

EXPLANATION

PAM (Pluggable Authentication Modules) is the Linux authentication framework. `pam_faillock` tracks failed authentication attempts per user. `deny=3` locks after 3 failures, `unlock_time=300` (seconds) auto-unlocks after 5 minutes. `pam_pwquality` enforces password complexity: `minlen=12` sets minimum length, `ucredit=-1` requires at least 1 uppercase, `dcridit=-1` requires 1 digit, `ocredit=-1` requires 1 special character. The 'requisite' keyword means failure immediately stops authentication (vs 'required' which continues checking remaining modules). Always test PAM changes with a second session open to avoid lockout.

PRO TIP

NEVER edit PAM files without a root session open in another terminal. A syntax error can lock you out completely.

REAL WORLD

PAM configuration is the first layer of authentication hardening in CIS Level 2 benchmarks and DISA STIG compliance.

15. umask and Default Permissions Strategy

USER & PERMISSIONS

INTERMEDIATE

#15

TASK

Set the system-wide default umask to 027 for regular users and 077 for root. Explain the resulting permissions for new files and directories.

SOLUTION

```
# For all users - add to /etc/profile or /etc/bashrc:
echo 'umask 027' >> /etc/profile

# For root specifically - add to /root/.bashrc:
echo 'umask 077' >> /root/.bashrc

# Apply immediately in current session:
umask 027

# Test:
touch testfile && ls -la testfile      # Should show: -rw-r-----
mkdir testdir && ls -ld testdir        # Should show: drwxr-xr-x

# Verify:
umask
```

EXPLANATION

umask (user file creation mask) defines which permission bits are removed from new files/dirs. It works by bitwise AND with the complement: $\text{permissions} = \text{mode} \& \sim\text{umask}$. With umask 027: files get 640 (rw-r-----) because $666 \& \sim 027 = 640$; directories get 750 (rwxr-xr-x) because $777 \& \sim 027 = 750$. With umask 077: files get 600 (rw-----), directories get 700 (rwx-----). umask 027 means: owner full access, group read-only, others nothing — appropriate for multi-user servers. umask 077 for root prevents accidental group/world readable root files.

PRO TIP

Use `stat -c '%a %n' filename` to check file permissions in octal. For the current umask: `umask -S` shows symbolic format.

REAL WORLD

Default umask settings are part of OS hardening benchmarks. Misconfigured umask is a common finding in security audits.

04

Process Management

5 assignments | Assignments 16-20

16. Process Tree Visualization and Analysis

PROCESS MANAGEMENT

INTERMEDIATE

#16

TASK

Display a complete process tree showing parent-child relationships, resource usage, and identify zombie/orphan processes.

SOLUTION

```
# Full process tree with formatting
ps auxf

# Better: pstree with PIDs and args
pstree -p -a -u

# Find zombie processes (Z state)
ps aux | awk '$8 == "Z" {print "Zombie PID:", $2, "PPID:", $3, "CMD:", $11}'

# Find orphaned processes (PPID=1)
ps -eo pid,ppid,stat,cmd | awk '$2==1 && $1!=1 {print "Orphan:", $0}'

# Kill zombie's parent to force reaping:
kill -SIGCHLD <parent_pid>

# Or kill parent if it's not a system process:
kill -9 <parent_pid>
```

EXPLANATION

ps auxf shows the ASCII tree with f=forest mode. pstree creates a visual tree: -p shows PIDs, -a shows command arguments, -u shows owning usernames. Zombie processes (state Z) have completed execution but their exit status hasn't been collected by the parent (wait() not called). They hold a PID but no resources — dangerous only if they accumulate (exhausts PID space). Orphan processes have their parent set to PID 1 (init/systemd) when the original parent dies. SIGCHLD sent to a parent forces it to reap zombies. Killing a parent causes init to adopt and reap its zombie children.

PRO TIP

Use `pidstat -p ALL 1` from `sysstat` for per-process CPU/memory statistics over time, useful for identifying resource-hungry processes.

REAL WORLD

Zombie processes in containerized environments often indicate poorly written Dockerfile ENTRYPOINT scripts that don't handle signals correctly.

17. Systemd Service Creation from Scratch

PROCESS MANAGEMENT

ADVANCED

#17

TASK

Create a systemd service for a Python Flask application that: auto-restarts on failure, starts after networking, has resource limits, and logs to journald.

SOLUTION

```
# Create /etc/systemd/system/flaskapp.service:
cat > /etc/systemd/system/flaskapp.service << 'EOF'
[Unit]
Description=Flask Application Server
After=network.target
Requires=network.target

[Service]
Type=simple
User=appuser
Group=appgroup
WorkingDirectory=/opt/flaskapp
EnvironmentFile=/opt/flaskapp/.env
ExecStart=/opt/flaskapp/venv/bin/python app.py
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure
RestartSec=5s
StartLimitInterval=60s
StartLimitBurst=3

# Security & Resource Limits
NoNewPrivileges=true
PrivateTmp=true
LimitNOFILE=65536
MemoryMax=512M
CPUQuota=80%

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now flaskapp
journalctl -u flaskapp -f
```

EXPLANATION

[Unit] section: After= sets start order, Requires= makes the dependency hard (service won't start if network fails). [Service]: Type=simple means the process doesn't fork. EnvironmentFile loads env vars from a file (keeps secrets out of the unit file). Restart=on-failure restarts if exit code is non-zero (not on intentional stop). RestartSec waits 5s before restarting. StartLimitBurst=3 with StartLimitInterval=60s means max 3 restarts per minute before giving up. Security options: NoNewPrivileges prevents privilege escalation via setuid. PrivateTmp gives the service its own /tmp (prevents /tmp attacks). LimitNOFILE sets the open file limit. MemoryMax and CPUQuota use Linux cgroups for resource limits.

PRO TIP

Use `systemd-analyze security flaskapp.service` to get a security score and specific recommendations for hardening the unit file.

REAL WORLD

The standard method for running web applications, workers, and daemons in modern Linux deployments without Docker.

18. cgroups Resource Throttling

PROCESS MANAGEMENT

ADVANCED

#18

TASK

Using cgroups v2, limit a running process (PID 5678) to 50% CPU and 256MB RAM, and prevent it from being OOM-killed.

SOLUTION

```
# Check cgroup version
mount | grep cgroup

# cgroups v2 approach:
# Create a cgroup
mkdir /sys/fs/cgroup/limited_app

# Set CPU limit (50% = 50000 out of 100000 microseconds)
echo '50000 100000' > /sys/fs/cgroup/limited_app/cpu.max

# Set memory limit (256MB)
echo '268435456' > /sys/fs/cgroup/limited_app/memory.max

# Protect from OOM killer
echo '1000' > /sys/fs/cgroup/limited_app/memory.oom.group

# Add process to cgroup
echo 5678 > /sys/fs/cgroup/limited_app/cgroup.procs

# Verify
cat /sys/fs/cgroup/limited_app/cpu.max
cat /sys/fs/cgroup/limited_app/memory.current
```

EXPLANATION

cgroups (control groups) is the Linux kernel mechanism for resource management, forming the foundation of Docker and Kubernetes resource limits. In cgroups v2, everything is under a unified hierarchy at /sys/fs/cgroup. cpu.max accepts 'quota period' format — 50000/100000 means the process can use 50ms of CPU per 100ms period (50%). memory.max limits RAM; the value is in bytes (256*1024*1024=268435456). Adding a PID to cgroup.procs moves it into that cgroup, applying all limits immediately. OOM protection with oom.group ensures related processes are treated together during memory pressure.

PRO TIP

systemd automatically creates cgroups for each service. Inspect them with: `systemctl status <service>` shows the cgroup path, then `cat /sys/fs/cgroup/<path>/memory.current`

REAL WORLD

Understanding cgroups is essential for Kubernetes debugging — every Pod and container runs in a cgroup hierarchy.

19. Signal Handling in Shell Scripts

PROCESS MANAGEMENT

ADVANCED

#19

TASK

Write a robust script that handles SIGTERM/SIGINT gracefully, cleans up temp files, releases locks, and reports its cleanup status before exiting.

SOLUTION

```
#!/bin/bash
LOCKFILE="/var/run/myapp.lock"
TMPDIR="/tmp/myapp_$$"
PID=$$

# Cleanup function
cleanup() {
    local exit_code=$?
    local signal=$1
    echo "[$(date)] Received signal: $signal (exit: $exit_code)"

    # Release lock
    [ -f "$LOCKFILE" ] && rm -f "$LOCKFILE" && echo "Lock released"

    # Clean temp files
    [ -d "$TMPDIR" ] && rm -rf "$TMPDIR" && echo "Temp dir cleaned"

    echo "[$(date)] Cleanup complete. Exiting."
    exit $exit_code
}

# Register signal handlers
trap 'cleanup SIGTERM' SIGTERM
trap 'cleanup SIGINT' SIGINT
trap 'cleanup EXIT' EXIT

# Acquire lock (prevent multiple instances)
if [ -f "$LOCKFILE" ]; then
    echo "Another instance running (PID: $(cat $LOCKFILE)). Exiting."
    exit 1
fi
echo $PID > "$LOCKFILE"

# Create temp working directory
mkdir -p "$TMPDIR"

# Main work
echo "Starting work (PID: $PID)"
sleep 100 # Simulate long-running task
```

EXPLANATION

The trap builtin registers handlers for signals and events. trap 'command' SIGNAL executes the command when the signal is received. EXIT is a pseudo-signal that fires on any exit (normal, error, or signal). The \$\$ variable holds the script's PID, used to create unique temp directories and lock files. The cleanup function captures the exit code before it changes, removes the lock file and temp dir, and exits with the original code. Single-instance locking via a PID file prevents duplicate runs — on startup, if the lock file exists and the PID is still running, the script aborts. This pattern is essential for cron jobs and background daemons.

PRO TIP

Use flock for more robust locking: `exec 200>/var/run/myapp.lock && flock -n 200 || exit 1`. It automatically releases the lock when the script exits, even on crash.

REAL WORLD

Every production shell script should implement signal handling. Kubernetes sends SIGTERM before SIGKILL — you have 30 seconds for graceful shutdown.

20. CPU Affinity and Process Pinning

PROCESS MANAGEMENT**ADVANCED**

#20

TASK

Pin a CPU-intensive process to cores 2 and 3 only, set its scheduling priority to real-time, and verify the affinity settings.

SOLUTION

```
# Start new process with CPU affinity (cores 2 and 3)
taskset -c 2,3 ./cpu_intensive_app &
APP_PID=$!

# Or set affinity on existing process (PID 9999)
taskset -c 2,3 -p 9999

# Set real-time scheduling priority (requires root)
chrt -f -p 50 $APP_PID # SCHED_FIFO with priority 50
# Or round-robin: chrt -r -p 50 $APP_PID

# Set nice value for non-RT priority adjustment (-20 to 19)
renice -n -10 -p $APP_PID

# Verify
taskset -c -p $APP_PID
chrt -p $APP_PID
ps -o pid,ni,pri,psr,cmd -p $APP_PID
```

EXPLANATION

taskset sets or retrieves CPU affinity — which cores a process is allowed to run on. -c accepts a comma-separated list or range (0-3) of CPU cores. CPU pinning improves cache locality and is critical for latency-sensitive applications. chrt sets the scheduling policy: SCHED_FIFO (-f) is real-time first-in-first-out where the process runs until it yields or a higher-priority RT process preempts it. Priority range for RT scheduling is 1-99 (higher = more priority). renice adjusts nice value for SCHED_OTHER (normal processes): -20 is highest priority, 19 is lowest. psr in ps output shows which CPU core the process is currently running on.

PRO TIP

Use numactl for NUMA-aware CPU and memory binding on multi-socket servers: `numactl --cpunodebind=0 --membind=0 ./app` pins to NUMA node 0.

REAL WORLD

Used for high-frequency trading systems, real-time audio processing, and database server tuning where latency and CPU cache efficiency are critical.

05

Networking

5 assignments | Assignments 24-28

21. Advanced iptables NAT and Port Forwarding

NETWORKING

ADVANCED

#21

TASK

Configure iptables to: forward port 8080 on the host to port 80 on an internal server at 192.168.1.10, set up masquerading for an internal network, and rate-limit SSH to 3 connections per minute.

SOLUTION

```
# Enable IP forwarding
echo 1 > /proc/sys/net/ipv4/ip_forward

# DNAT: forward external port 8080 to internal 192.168.1.10:80
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8080 -j DNAT --to-destination 192.168.1.10:80

# Allow the forwarded traffic
iptables -A FORWARD -p tcp -d 192.168.1.10 --dport 80 -j ACCEPT
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# Masquerade (SNAT) for internal network
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE

# Rate-limit SSH: max 3 new connections per minute
iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
-m recent --set --name SSH
iptables -A INPUT -p tcp --dport 22 -m state --state NEW \
-m recent --update --seconds 60 --hitcount 4 --name SSH -j DROP

# Save
iptables-save > /etc/iptables/rules.v4
```

EXPLANATION

iptables has multiple tables: filter (default), nat, mangle. NAT table PREROUTING chain intercepts packets before routing. DNAT (Destination NAT) rewrites the destination IP/port — traffic arriving on port 8080 gets redirected to the internal server. MASQUERADE automatically replaces the source IP with the outgoing interface's IP (suitable when external IP is dynamic). The FORWARD chain controls routed traffic between interfaces. Rate limiting uses the 'recent' module: --set adds the source IP to a named list, --update checks if the IP has more than hitcount (4) hits in seconds (60). ip_forward must be enabled for packet routing to work.

PRO TIP

Use nft (nftables) for new deployments — it replaces iptables with a cleaner syntax and atomic rule updates: `nft -f /etc/nftables.conf`

REAL WORLD

This configuration pattern is the foundation of bastion hosts, NAT gateways in AWS VPCs, and on-premise network edge devices.

22. Network Namespace Isolation

NETWORKING

ADVANCED

#22

TASK

Create an isolated network namespace, add a virtual ethernet pair to connect it to the host network, assign IP addresses, and test connectivity.

SOLUTION

```
# Create namespace
ip netns add isolated_ns

# Create veth pair (virtual ethernet)
ip link add veth0 type veth peer name veth1

# Move veth1 into the namespace
ip link set veth1 netns isolated_ns

# Configure host side
ip addr add 10.0.0.1/24 dev veth0
ip link set veth0 up

# Configure namespace side
ip netns exec isolated_ns ip addr add 10.0.0.2/24 dev veth1
ip netns exec isolated_ns ip link set veth1 up
ip netns exec isolated_ns ip link set lo up
ip netns exec isolated_ns ip route add default via 10.0.0.1

# Test
ip netns exec isolated_ns ping -c3 10.0.0.1

# Run a process in namespace
ip netns exec isolated_ns bash
```

EXPLANATION

Linux network namespaces provide isolated network stacks — each has its own interfaces, routing table, firewall rules, and sockets. This is exactly how Docker implements container networking. veth (virtual ethernet) pairs work like a network cable connecting two namespaces — what goes in one end comes out the other. The pair is split: one end stays in the root namespace (veth0), the other moves into the new namespace (veth1). `ip netns exec` runs commands inside a specific namespace. `lo` must be explicitly brought up in each namespace. The default route inside the namespace points to the host-side veth as its gateway.

PRO TIP

Explore a container's network namespace: `ip netns list` won't show Docker namespaces — use `nsenter -t <container_pid> -n ip addr` instead.

REAL WORLD

Deep understanding of network namespaces is essential for debugging Kubernetes CNI (Container Network Interface) issues and custom networking setups.

23. SSL/TLS Certificate Management

NETWORKING

ADVANCED

#23

TASK

Generate a self-signed certificate, create a Certificate Signing Request (CSR) for production, verify certificate details, and check expiry dates across multiple servers.

SOLUTION

```
# Generate private key and self-signed cert
openssl req -x509 -newkey rsa:4096 -keyout server.key -out server.crt \
  -days 365 -nodes -subj '/CN=devopsshack.com/O=DevOps Shack/C=IN'

# Generate key + CSR for production CA signing
openssl req -newkey rsa:4096 -keyout prod.key -out prod.csr -nodes \
  -subj '/CN=devopsshack.com'

# Add Subject Alternative Names (required for modern browsers)
openssl req -newkey rsa:4096 -keyout san.key -out san.csr -nodes \
  -config <(cat /etc/ssl/openssl.cnf \
    <(printf
' [SAN]\nsubjectAltName=DNS:devopsshack.com,DNS:www.devopsshack.com,DNS:api.devopsshack.com'))
# Inspect certificate
openssl x509 -in server.crt -text -noout | grep -E 'Subject:|Not|Alt'

# Check expiry across servers
for host in web1 web2 web3; do
  echo -n "$host: "
  echo | openssl s_client -connect $host:443 2>/dev/null | \
    openssl x509 -noout -enddate
done
```

EXPLANATION

openssl req generates certificate requests and self-signed certificates. -newkey rsa:4096 creates a 4096-bit RSA key. -nodes (no DES) means the private key is not password-protected (required for automated server restarts). -x509 makes it self-signed instead of a CSR. -subj sets the Distinguished Name inline. Subject Alternative Names (SANs) are required by browsers since 2017 — a certificate without SANs will show warnings even if CN matches. For production, send the .csr to a CA (Let's Encrypt, DigiCert). openssl s_client connects to live servers to retrieve and inspect their certificates. The expiry check loop is useful for catching certificates about to expire.

PRO TIP

Use certbot for automatic Let's Encrypt certificates: `certbot --nginx -d devopsshack.com -d www.devopsshack.com`. Autorenewal is handled by a systemd timer.

REAL WORLD

Certificate management is a critical operational task. Expired certificates have caused major outages at companies like Microsoft, LinkedIn, and various banks.

24. Network Bonding and Teaming

NETWORKING

ADVANCED

#24

TASK

Configure network bonding in LACP (802.3ad) mode combining eth1 and eth2 for both redundancy and throughput, and verify link status.

SOLUTION

```
# Install bonding utilities
apt-get install -y ifenslave

# Load bonding module with LACP
modprobe bonding mode=4 miimon=100 lacp_rate=fast

# Create bond interface (via /etc/network/interfaces on Debian)
cat >> /etc/network/interfaces << 'EOF'
auto bond0
iface bond0 inet static
    address 192.168.1.100
    netmask 255.255.255.0
    gateway 192.168.1.1
    bond-slaves eth1 eth2
    bond-mode 4
    bond-miimon 100
    bond-lacp-rate fast
    bond-xmit-hash-policy layer3+4
EOF

# For NetworkManager (modern approach):
nmcli con add type bond ifname bond0 bond.options 'mode=active-backup,miimon=100'
nmcli con add type ethernet ifname eth1 master bond0
nmcli con add type ethernet ifname eth2 master bond0

# Verify bond status
cat /proc/net/bonding/bond0
```

EXPLANATION

Network bonding combines multiple physical NICs into one logical interface. Mode 4 (802.3ad LACP) uses Link Aggregation Control Protocol to dynamically negotiate with the switch for active-active bonding — both interfaces carry traffic simultaneously (increased throughput) and failover is automatic if one fails. `miimon=100` checks link state every 100ms. `lacp_rate=fast` increases LACP PDU exchange to every 1 second (vs 30 sec slow). `layer3+4` hashing uses IP address + port for load distribution, ensuring packets from the same TCP connection go through the same interface (prevents out-of-order delivery). The switch must also be configured for LACP port channels.

PRO TIP

Mode 1 (active-backup) is simpler and works without switch configuration — one interface is active, the other is standby. Use for simple redundancy.

REAL WORLD

Standard configuration for database servers, storage systems, and any server where network availability is critical.

25. Advanced SSH Tunneling and Port Forwarding

NETWORKING

ADVANCED

#25

TASK

Set up: (1) a local forward to access a remote MySQL database, (2) a reverse tunnel to expose a local dev server publicly, and (3) a dynamic SOCKS proxy.

SOLUTION

```
# 1. LOCAL FORWARD: Access remote MySQL on localhost:3307
ssh -L 3307:127.0.0.1:3306 user@jump-server.com -N -f
# Now: mysql -h 127.0.0.1 -P 3307

# 2. REVERSE TUNNEL: Expose local:8080 on remote:9090
ssh -R 9090:localhost:8080 user@public-server.com -N -f
# Public users: curl http://public-server.com:9090

# 3. DYNAMIC SOCKS5 PROXY: Route browser traffic through server
ssh -D 1080 user@remote-server.com -N -f
# Configure browser to use SOCKS5 proxy at 127.0.0.1:1080

# Persistent tunnels with autossh:
autossh -M 0 -f -N -L 3307:db-server:3306 user@jump-server.com \
-o ServerAliveInterval=30 -o ServerAliveCountMax=3

# SSH config for convenience (~/.ssh/config):
# Host mysql-tunnel
#   HostName jump-server.com
#   LocalForward 3307 127.0.0.1:3306
#   ServerAliveInterval 30
```

EXPLANATION

SSH tunneling encrypts traffic through an SSH connection. Local forward (-L local_port:target_host:target_port): binds a port on your machine; connections are forwarded through the SSH server to the target (useful for accessing services behind firewalls). Reverse tunnel (-R remote_port:local_host:local_port): binds a port on the remote server; connections to it are forwarded to your local machine (useful for exposing development servers). Dynamic (-D port): creates a SOCKS proxy on the local port; applications can route all traffic through it. -N means no command execution (tunnel only). -f forks to background. autossh automatically restarts tunnels that drop. ServerAliveInterval sends keepalive packets to prevent firewall timeouts.

PRO TIP

For production use cases, consider Cloudflare Tunnel (cloudflared) or Tailscale as more robust alternatives to SSH reverse tunnels.

REAL WORLD

SSH tunnels are essential for DevOps engineers to securely access databases, internal services, and debug production systems without exposing ports.

06

Disk & Storage

5 assignments | Assignments 26-30

26. LVM (Logical Volume Manager) Setup and Expansion

DISK & STORAGE

ADVANCED

#26

TASK

Create an LVM setup from two disks (/dev/sdb, /dev/sdc), create a logical volume, format it, and extend it online without downtime.

SOLUTION

```
# Initialize physical volumes
pvcreate /dev/sdb /dev/sdc

# Create volume group
vgcreate datavg /dev/sdb /dev/sdc

# Create 50GB logical volume
lvcreate -L 50G -n datalv datavg

# Format and mount
mkfs.xfs /dev/datavg/datalv
mount /dev/datavg/datalv /data

# Add fstab entry
echo '/dev/datavg/datalv /data xfs defaults 0 0' >> /etc/fstab

# Later: extend by 20GB (online, no unmount needed for XFS)
lvextend -L +20G /dev/datavg/datalv
xfs_growfs /data

# Add new disk /dev/sdd to existing VG
pvcreate /dev/sdd
vgextend datavg /dev/sdd

# Check everything
pvs && vgs && lvs
```

EXPLANATION

LVM provides an abstraction layer between physical disks and filesystems. Physical Volumes (PV) are raw disks/partitions initialized for LVM. Volume Groups (VG) pool multiple PVs into one storage pool. Logical Volumes (LV) are carved out of the VG pool and behave like virtual partitions. The major advantage: LVs can be resized online without unmounting (with XFS or ext4). `lvextend` adds space to the LV. `xfs_growfs` expands the filesystem to fill the new LV size (use `resize2fs` for ext4). `vgextend` adds a new PV to an existing VG, immediately adding its space to the pool. `pvs/vgs/lvs` show status of each layer.

PRO TIP

Take LVM snapshots before risky operations: `lvcreate -L 5G -s -n snap /dev/datavg/datalv`.
Revert with: `lvconvert --merge /dev/datavg/snap`

REAL WORLD

LVM is the standard storage abstraction in RHEL/CentOS. Understanding it is required for database storage management and live system maintenance.

27. RAID Configuration with mdadm

DISK & STORAGE

ADVANCED

#27

TASK

Create a software RAID-5 array with 3 disks (/dev/sdb, /dev/sdc, /dev/sdd), monitor its status, simulate a disk failure, and rebuild with a replacement disk.

SOLUTION

```
# Create RAID-5 array
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb /dev/sdc /dev/sdd

# Wait for sync (or check progress)
watch cat /proc/mdstat

# Create filesystem and mount
mkfs.ext4 /dev/md0
mount /dev/md0 /mnt/raid5

# Save RAID config
mdadm --detail --scan >> /etc/mdadm/mdadm.conf
update-initramfs -u

# Simulate disk failure
mdadm --manage /dev/md0 --fail /dev/sdc

# Remove failed disk
mdadm --manage /dev/md0 --remove /dev/sdc

# Add replacement disk
mdadm --manage /dev/md0 --add /dev/sde

# Monitor rebuild
mdadm --detail /dev/md0
cat /proc/mdstat
```

EXPLANATION

Software RAID uses the mdadm tool and the Linux md (multiple devices) driver. RAID-5 requires minimum 3 disks and provides one disk's worth of redundancy — a distributed parity block allows recovery from any single disk failure while using N-1 disks for actual storage. /proc/mdstat shows real-time sync progress. The config must be saved to /etc/mdadm/mdadm.conf so the array auto-assembles at boot; update-initramfs includes it in the boot image. To simulate failure: --fail marks a disk as failed, triggering degraded mode. --remove removes it from the array. --add adds a replacement, immediately beginning automatic rebuild. RAID-5 remains readable/writable in degraded mode but has zero redundancy until rebuild completes.

PRO TIP

RAID is not a backup. A RAID-5 array doesn't protect against accidental deletion, filesystem corruption, or simultaneous multi-disk failure. Always pair RAID with backups.

REAL WORLD

Software RAID is common in cost-sensitive on-premise deployments. Understanding mdadm is essential for managing legacy and bare-metal servers.

28. ZFS Filesystem Operations

DISK & STORAGE

ADVANCED

#28

TASK

Create a ZFS pool with a mirror, set quotas, enable compression, take a snapshot, send it to a remote server, and roll back to it.

SOLUTION

```
# Create mirrored ZFS pool
zpool create datapool mirror /dev/sdb /dev/sdc

# Create dataset with compression and quotas
zfs create -o compression=lz4 -o quota=100G datapool/appdata

# Set additional properties
zfs set atime=off datapool/appdata # Disable access time updates
zfs set recordsize=16K datapool/appdata # Optimize for databases

# Take snapshot
zfs snapshot datapool/appdata@2024-11-01

# List snapshots
zfs list -t snapshot

# Send snapshot to remote (incremental after first full send)
zfs send datapool/appdata@2024-11-01 | ssh backup-server zfs recv backuppool/appdata

# Rollback
zfs rollback datapool/appdata@2024-11-01

# Pool status
zpool status datapool
zfs get all datapool/appdata
```

EXPLANATION

ZFS is a combined filesystem and volume manager with built-in RAID (raidz), snapshots, checksumming, and compression. `zpool create` defines the storage pool and redundancy level (mirror = RAID-1). `zfs create` carves datasets (like subdirectories with independent properties) from the pool. `lz4` compression has minimal CPU overhead with 2-3x compression ratios — always worth enabling. Snapshots are instant, space-efficient copy-on-write checkpoints — they consume space only as data changes. `zfs send` generates a binary stream of a snapshot; piping over SSH to `zfs recv` enables efficient replication. Incremental sends (-i) only transfer changes since the last snapshot. Rollback reverts the dataset to the snapshot state.

PRO TIP

ZFS send/recv is the gold standard for data replication. Combine with `zfs send -R` for recursive replication of all child datasets.

REAL WORLD

ZFS is the default filesystem on FreeBSD, TrueNAS, and Oracle Solaris. It's increasingly used on Linux for NAS servers and high-integrity storage requirements.

29. Disk Benchmarking and I/O Analysis

DISK & STORAGE

INTERMEDIATE

#29

TASK

Benchmark `/dev/sdb` for sequential read/write and random I/O performance. Identify I/O bottlenecks in a running system.

SOLUTION

```
# Sequential read/write benchmark with dd
dd if=/dev/zero of=/tmp/testfile bs=1G count=1 oflag=direct
dd if=/tmp/testfile of=/dev/null bs=1G iflag=direct

# Advanced benchmark with fio
fio --name=seq-write --ioengine=libaio --iodepth=32 \
    --rw=write --bs=1M --size=1G --runtime=30 \
    --filename=/tmp/fiotest --direct=1 --numjobs=1
fio --name=rand-4k --ioengine=libaio --iodepth=64 \
    --rw=randread --bs=4k --size=1G --runtime=30 \
    --filename=/tmp/fiotest --direct=1

# Real-time I/O monitoring
iostat -x 1 5 # 5 samples, 1 sec interval
iotop -a      # Per-process I/O (like top for disk)

# Find processes causing I/O
pidstat -d 1 10
```

EXPLANATION

`dd` with `oflag=direct` bypasses the page cache (direct I/O), giving true disk performance rather than memory speed. `bs=1G` tests large sequential operations. `fio` is the standard storage benchmarking tool: `ioengine=libaio` uses Linux asynchronous I/O for realistic performance. `iodepth` controls the I/O queue depth (higher = better for SSDs/NVMe). `rw=write/read/randread/randwrite` defines the access pattern. `bs=4k` simulates database-style random I/O. `direct=1` bypasses cache. `iostat -x` shows extended device statistics: `%util` shows how busy the device is, `await` is average I/O wait time, `r/s` and `w/s` are operation rates. `iotop` (like `top` for I/O) shows per-process I/O in real time.

PRO TIP

Key metrics to watch: `%util >80%` means I/O saturation; `await >10ms` on SSD indicates queue buildup; very high `iowait%` in `top` means processes are blocked waiting for I/O.

REAL WORLD

Disk benchmarking is the first step in sizing infrastructure, debugging database slowness, and comparing storage tiers in cloud environments.

30. Automated Incremental Backup with rsync

DISK & STORAGE

INTERMEDIATE

#30

TASK

Create an incremental backup solution using rsync that maintains 30 daily snapshots using hard links (like Time Machine), with checksumming and email on completion.

SOLUTION

```
#!/bin/bash
SRC="/var/www"
DEST="/backup"
DATE=$(date +%Y-%m-%d)
LATEST="$DEST/latest"
LOG="/var/log/rsync_backup.log"
mkdir -p "$DEST/$DATE"
# Incremental backup using hard links from previous backup
rsync -av --delete --checksum \
  --link-dest="$LATEST" \
  "$SRC/" "$DEST/$DATE/"
RESULT=$?
# Update latest symlink
rm -f "$LATEST"
ln -s "$DEST/$DATE" "$LATEST"
# Remove backups older than 30 days
find "$DEST" -maxdepth 1 -type d -mtime +30 -exec rm -rf {} \;
# Log and notify
if [ $RESULT -eq 0 ]; then
  echo "[${DATE}] Backup SUCCESS" >> "$LOG"
  SIZE=$(du -sh "$DEST/$DATE" | cut -f1)
  echo "Backup done. Size: $SIZE" | mail -s "Backup OK" admin@devopsshack.com
else
  echo "[${DATE}] Backup FAILED (exit: $RESULT)" >> "$LOG"
  echo "BACKUP FAILED on $DATE" | mail -s "BACKUP FAILURE" admin@devopsshack.com
fi
```

EXPLANATION

rsync with `--link-dest` creates hard links to unchanged files from the reference directory instead of copying them, making each daily snapshot appear full while only consuming space for changed files — identical to macOS Time Machine. `--delete` removes files from destination that no longer exist in source (synchronized mirror). `--checksum` compares file content (not just size/timestamp) for accuracy, slower but detects bit-rot. The `$RESULT` captures rsync exit code: 0=success, non-zero=failure. The latest symlink always points to the most recent successful backup, providing a stable reference for `--link-dest`. Removing directories older than 30 days automatically manages retention.

PRO TIP

For remote backups, add SSH: `rsync -av --link-dest=... -e 'ssh -i /root/.ssh/backup_key' src/user@backup-server:/path/`. Combine with a read-only SSH key for security.

REAL WORLD

This pattern is used in enterprise backup strategies alongside tools like Bacula, Amanda, or commercial solutions for 3-2-1 backup compliance.

07

System Administration

5 assignments | Assignments 24-28

31. Kernel Module Management

SYSTEM ADMINISTRATION

ADVANCED

#31

TASK

Load, configure, persist, and troubleshoot Linux kernel modules. Blacklist the 'pcspkr' module and load 'br_netfilter' with specific parameters at boot.

SOLUTION

```
# List loaded modules
lsmod

# Get info about a module
modinfo br_netfilter

# Load module manually
modprobe br_netfilter

# Load with parameters
modprobe dummy numdummies=2

# Persist modules at boot
echo 'br_netfilter' > /etc/modules-load.d/k8s.conf

# Set module parameters persistently
cat > /etc/modprobe.d/br_netfilter.conf << 'EOF'
options br_netfilter nf_call_iptables=1
EOF

# Blacklist a module
cat > /etc/modprobe.d/blacklist-pcspkr.conf << 'EOF'
blacklist pcspkr
install pcspkr /bin/false
EOF

# Rebuild initramfs to include changes
update-initramfs -u -k all

# Verify
lsmod | grep br_netfilter
cat /sys/module/br_netfilter/parameters/nf_call_iptables
```

EXPLANATION

Linux kernel modules are loadable code extensions (.ko files) that add driver or feature support without rebooting. Lsmod reads /proc/modules to show loaded modules. modinfo shows metadata: version, parameters, dependencies. modprobe loads a module and its dependencies (better than insmod). Module parameters can be passed at load time. /etc/modules-load.d/*.conf contains modules to load at boot. /etc/modprobe.d/*.conf sets persistent parameters and rules. Blacklisting prevents a module from loading, but 'blacklist' alone can be overridden by dependencies; 'install modulename /bin/false' completely blocks it. update-initramfs embeds the configuration into the boot image so changes persist across reboots.

PRO TIP

For a persistent dashboard, run it with `watch -n 5 ./dashboard.sh`. Or install Netdata for a zero-config web-based real-time dashboard.

REAL WORLD

Quick health checks are the first step in incident response. Having this as a script on all servers speeds up initial triage significantly.

33. Audit Framework with auditd

SYSTEM ADMINISTRATION

ADVANCED

#33

TASK

Configure auditd to monitor: all writes to `/etc/passwd` and `/etc/shadow`, all `sudo` command executions, and all failed file accesses. Generate a compliance report.

SOLUTION

```
# Install and start auditd
apt-get install -y auditd
systemctl enable --now auditd

# Add audit rules
auditctl -w /etc/passwd -p wa -k identity_changes
auditctl -w /etc/shadow -p wa -k identity_changes
auditctl -w /etc/sudoers -p wa -k sudoers_changes

# Monitor sudo execution (execve syscall for sudo)
auditctl -a always,exit -F arch=b64 -S execve -F path=/usr/bin/sudo -k sudo_usage

# Log failed file accesses
auditctl -a always,exit -F arch=b64 -S open -F exit=-EACCES -k access_denied
auditctl -a always,exit -F arch=b64 -S open -F exit=-EPERM -k access_denied

# Persist rules to /etc/audit/rules.d/audit.rules
auditctl -l > /etc/audit/rules.d/main.rules

# Generate compliance report
ausearch -k identity_changes --start today | aureport -f
ausearch -k sudo_usage --start this-week | aureport -u
ausearch -k access_denied --start today | aureport -a
```

EXPLANATION

auditd is the Linux kernel audit daemon — it records system calls and file access events to `/var/log/audit/audit.log`. auditctl adds rules at runtime. `-w` watches a file or directory. `-p` specifies permissions to monitor: `w`=write, `a`=attribute change, `r`=read, `x`=execute. `-k` assigns a key (tag) for easy searching later. For syscall auditing (`-a`), `always,exit` means log when the syscall exits (completed). `arch=b64` specifies 64-bit syscalls. `-S` specifies the syscall. `-F` adds filters. `exit=-EACCES/-EPERM` catches permission denied errors. `ausearch` queries logs by key/time. `aureport` generates summary reports: `-f`=file activity, `-u`=user activity, `-a`=authentication events.

PRO TIP

Use auditd with AIDE (Advanced Intrusion Detection Environment) for comprehensive file integrity monitoring. AIDE takes a database snapshot; later runs compare against it.

REAL WORLD

auditd configuration is a mandatory requirement for PCI-DSS, HIPAA, SOC2, and all major compliance frameworks. Its logs are the forensic record for security incidents.

34. Systemd Journal Advanced Management

SYSTEM ADMINISTRATION	INTERMEDIATE	#34
-----------------------	--------------	-----

TASK

Configure journal to persist across reboots, limit storage to 500MB, extract specific time-range logs, filter by priority, and export for external analysis.

SOLUTION

```
# Persist journals (default is /run, lost on reboot)
mkdir -p /var/log/journal
systemd-tmpfiles --create --prefix /var/log/journal
killall -USR1 systemd-journald

# Configure limits in /etc/systemd/journald.conf
cat >> /etc/systemd/journald.conf << 'EOF'
[Journal]
Storage=persistent
SystemMaxUse=500M
SystemKeepFree=100M
MaxFileSec=1month
Compress=yes
EOF

systemctl restart systemd-journald

# Query examples
journalctl -p err -b # Errors since last boot
journalctl --since '2024-11-01 00:00' --until '2024-11-02 00:00'
journalctl -u nginx --since '1 hour ago' -f
journalctl -p warning..crit -n 100 # Warning to critical, last 100

# Export for analysis
journalctl --since yesterday -o json > /tmp/journal_export.json
journalctl --since yesterday -o json | jq 'select(.__PRIORITY <= "3")'

# Check journal disk usage
journalctl --disk-usage
```

EXPLANATION

By default, systemd journals are stored in /run/log/journal/ (RAM, cleared on reboot). Creating /var/log/journal/ switches to persistent storage. journald.conf SystemMaxUse limits total disk usage; SystemKeepFree ensures minimum free space; MaxFileSec sets rotation period. Compress=yes uses lz4 compression. journalctl priority levels: 0=emergency, 1=alert, 2=critical, 3=error, 4=warning, 5=notice, 6=info, 7=debug. Range syntax warning..crit captures 4,3,2. -b shows logs from the current boot (use -b -1

for previous boot). JSON output format exposes all structured fields for processing with jq or sending to log aggregators like Elasticsearch.

PRO TIP

Forward journald logs to syslog-compatible systems: set ForwardToSyslog=yes in journald.conf. Or use systemd-journal-remote for network log forwarding.

REAL WORLD

Centralized log management is a core DevOps practice. journald's structured logging with JSON export integrates cleanly with ELK stack and Grafana Loki.

35. Boot Process Optimization and Troubleshooting

SYSTEM ADMINISTRATION**ADVANCED**

#35

TASK

Analyze the boot time breakdown, identify the slowest services, disable unnecessary services, and reduce boot time by at least 30%.

SOLUTION

```
# Analyze total boot time
systemd-analyze

# Detailed breakdown by service
systemd-analyze blame

# Generate boot timeline SVG
systemd-analyze plot > /tmp/boot_timeline.svg

# Find critical path (longest dependency chain)
systemd-analyze critical-chain

# Check unit dependencies
systemctl list-dependencies

# Find slow services (>3 seconds)
systemd-analyze blame | awk '$1 ~ /s$/ && $1+0 > 3 {print}'

# Disable unnecessary services
for svc in bluetooth cups avahi-daemon ModemManager; do
    systemctl disable --now $svc 2>/dev/null && echo "Disabled: $svc"
done

# Enable parallel startup where possible
# Edit /etc/systemd/system.conf:
# DefaultLimitNOFILE=65536

# Mask services completely (prevents accidental start)
systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target

# After changes:
systemctl daemon-reload
systemd-analyze # Compare new time
```

EXPLANATION

systemd-analyze shows three phases: firmware time (BIOS/UEFI), loader time (GRUB), kernel time, and userspace time. systemd-analyze blame lists all services by their initialization time — the longest ones are optimization targets. The critical chain shows the sequence of units with no parallel execution possible (the actual bottleneck path). Disabling Bluetooth, CUPS (printing), Avahi (mDNS/Bonjour), and ModemManager (GSM modems) is safe on servers where these are unused. Masking (vs disabling) completely prevents a unit from being started, even as a dependency. The SVG plot visualizes the parallel execution of services during boot.

PRO TIP

Check if systemd-resolved or NetworkManager is causing delays. On servers, replacing NetworkManager with systemd-networkd often reduces boot time by 5-10 seconds.

REAL WORLD

Boot optimization matters for auto-scaling instances that must be ready quickly. It's also critical for embedded systems and edge computing devices.

08

Shell Scripting

5 assignments | Assignments 26-40

36. Parallel Job Execution Framework

SHELL SCRIPTING

ADVANCED

#36

TASK

Write a script that processes 100 servers in parallel (max 10 at a time), runs a health check on each, collects results, and generates a summary report.

SOLUTION

```
#!/bin/bash
SERVERS=$(cat /etc/server_list.txt)
MAX_PARALLEL=10
RESULTS_DIR="/tmp/health_check_$$"
mkdir -p "$RESULTS_DIR"

check_server() {
    local server=$1
    local result_file="$RESULTS_DIR/$server"

    # Run checks with timeout
    if timeout 10 ssh -o ConnectTimeout=5 -o StrictHostKeyChecking=no \
        "$server" "uptime && df -h / && free -m" > "$result_file" 2>&1; then
        echo "OK" >> "$result_file"
    else
        echo "FAILED (exit: $?)" >> "$result_file"
    fi
}

export -f check_server
export RESULTS_DIR

# Run in parallel with max 10 concurrent jobs
printf '%s\n' "${SERVERS[@]}" | \
    xargs -P $MAX_PARALLEL -I {} bash -c 'check_server "$@"' _ {}

# Generate report
OK=0; FAIL=0
for server in "${SERVERS[@]}; do
    if grep -q "^OK$" "$RESULTS_DIR/$server" 2>/dev/null; then
        ((OK++))
    else
        echo "FAILED: $server"
        ((FAIL++))
    fi
done

echo "\nSummary: $OK OK, $FAIL FAILED"
rm -rf "$RESULTS_DIR"
```

EXPLANATION

xargs -P sets the maximum number of parallel processes. -I {} defines the placeholder for each input item. export -f exports the shell function so child processes spawned by xargs can access it. export RESULTS_DIR passes the variable to subprocesses. Each check runs in its own subshell, writes results

to a unique temp file, and exits. timeout prevents hanging on unresponsive servers. ssh -o ConnectTimeout=5 limits connection time. StrictHostKeyChecking=no prevents interactive prompts (use known_hosts in production). The \$\$ variable ensures unique directory names for concurrent runs. Results are aggregated after all parallel jobs complete.

PRO TIP

GNU parallel is a more powerful alternative: parallel --jobs 10 ssh {} 'uptime' ::: server_list.txt. It handles job output, retries, and progress much better than xargs.

REAL WORLD

Parallel execution patterns are essential for fleet management, mass deployments, and monitoring scripts across large server pools.

37. Configuration Template Engine

SHELL SCRIPTING**ADVANCED**

#37

TASK

Build a templating system that takes a config template with `{{VARIABLE}}` placeholders and an environment file, generates the final config, validates it, and deploys it.

SOLUTION

```
#!/bin/bash
TEMPLATE="$1"
ENV_FILE="$2"
OUTPUT="$3"
VALIDATOR="${4:-}" # Optional validator command

# Validate inputs
[ -z "$TEMPLATE" ] || [ -z "$ENV_FILE" ] || [ -z "$OUTPUT" ] && {
    echo "Usage: $0 <template> <env_file> <output> [validator]"
    exit 1
}
[ -f "$TEMPLATE" ] || { echo "Template not found: $TEMPLATE"; exit 1; }
[ -f "$ENV_FILE" ] || { echo "Env file not found: $ENV_FILE"; exit 1; }

# Load env vars
set -a
source "$ENV_FILE"
set +a

# Process template: replace {{VAR}} with env value
result=$(cat "$TEMPLATE")
missing_vars=()

# Find all placeholders
while IFS= read -r var; do
    value="${!var}"
    if [ -z "$value" ]; then
        missing_vars+=("$var")
    else
        result="${result//\{\{${var}\}\}/$value}"
    fi
done < <(grep -oP '(?<=\{\}\w+(?=\}\})' "$TEMPLATE" | sort -u)
```

```
# Fail if any variables missing
[ ${#missing_vars[@]} -gt 0 ] && {
    echo "ERROR: Missing variables: ${missing_vars[*]}"
    exit 2
}

# Write output
echo "$result" > "$OUTPUT"

# Run validator if provided
if [ -n "$VALIDATOR" ]; then
    $VALIDATOR "$OUTPUT" || { echo "Validation failed"; exit 3; }
fi

echo "Config generated: $OUTPUT"
```

EXPLANATION

set -a marks all variables for export so sourced env file variables are available. set +a restores normal behavior. Bash parameter expansion \${!var} performs indirect reference — it gets the value of the variable whose name is stored in \$var. The string replacement \${result//{{\${var}}}/\$value} uses bash's built-in substitution (// = global). grep -oP with PCRE lookahead/lookbehind extracts only the variable names between {{ }}. The while loop with process substitution < <(...) avoids a subshell (variables persist). Missing variable detection prevents deploying a config with literal {{VARIABLE}} placeholders. The optional validator (e.g., nginx -t) verifies the generated config before deployment.

PRO TIP

For complex templates with conditionals and loops, use envsubst (from gettext) for simple substitution, or adopt Jinja2 via Python for production config management.

REAL WORLD

This pattern is the basis of Helm chart rendering, Ansible templates, and Docker entrypoint scripts that configure applications from environment variables.

38. Database Backup with Rotation and Encryption

SHELL SCRIPTING

ADVANCED

#38

TASK

Write a script that backs up all MySQL databases individually, encrypts each with GPG, rotates keeping 7 daily + 4 weekly backups, and verifies backup integrity.

SOLUTION

```
#!/bin/bash
BACKUP_DIR="/backup/mysql"
DATE=$(date +%Y%m%d_%H%M%S)
DAY_OF_WEEK=$(date +%u) # 1=Mon, 7=Sun
GPG_RECIPIENT="admin@devopsshack.com"
MYSQL_USER="backup_user"
MYSQL_PASS="${MYSQL_BACKUP_PASSWORD}" # From env variable
```

```

mkdir -p "$BACKUP_DIR/daily" "$BACKUP_DIR/weekly"

# Get all databases
databases=$(mysql -u"$MYSQL_USER" -p"$MYSQL_PASS" \
  -e "SHOW DATABASES;" | grep -Ev
  "^ (Database|information_schema|performance_schema|sys)$")

for db in $databases; do
  echo "Backing up: $db"
  backup_file="$BACKUP_DIR/daily/${db}_${DATE}.sql.gz.gpg"

  # Dump | Compress | Encrypt
  mysqldump -u"$MYSQL_USER" -p"$MYSQL_PASS" \
    --single-transaction --routines --triggers \
    "$db" | \
    gzip | \
    gpg --batch --yes --recipient "$GPG_RECIPIENT" \
    --output "$backup_file" --encrypt

  # Verify
  gpg --batch --yes --decrypt "$backup_file" | \
    gzip -t && echo "  Verified: $backup_file" || \
    echo "  VERIFICATION FAILED: $backup_file"
done

# Weekly backup on Sunday
[ "$DAY_OF_WEEK" -eq 7 ] && \
  cp -a "$BACKUP_DIR/daily/." "$BACKUP_DIR/weekly/"

# Rotate: keep 7 daily, 4 weekly (28 days)
find "$BACKUP_DIR/daily" -mtime +7 -delete
find "$BACKUP_DIR/weekly" -mtime +28 -delete

```

EXPLANATION

mysql SHOW DATABASES lists all databases; grep -Ev excludes system databases. mysqldump with --single-transaction creates a consistent snapshot without locking tables (for InnoDB). --routines includes stored procedures, --triggers includes triggers. The pipeline Dump | gzip | gpg compresses and encrypts in one pass without intermediate files (saves disk space and avoids unencrypted data touching disk). GPG asymmetric encryption means only the private key holder can decrypt — even if backups are stolen, data is protected. Verification with gzip -t (test) decrypts and checks gzip integrity. Weekly backups are a hard-copy of Sunday's daily set. find -delete removes old files for retention management.

PRO TIP

Use --master-data=2 in mysqldump for backups that include binary log position — essential for point-in-time recovery and setting up replication replicas.

REAL WORLD

Encrypted, rotating database backups are a non-negotiable requirement for any production system. Combine with offsite storage (S3, GCS) for disaster recovery.

39. Infrastructure Drift Detection

SHELL SCRIPTING

ADVANCED

#39

TASK

Write a script that captures a baseline of installed packages, running services, and open ports — then compares against the baseline to detect unauthorized changes.

SOLUTION

```
#!/bin/bash
BASELINE_DIR="/var/lib/baseline"
CURRENT_DIR="/tmp/baseline_check_$$"
REPORT="/var/log/drift_report_$(date +%Y%m%d).txt"
mkdir -p "$BASELINE_DIR" "$CURRENT_DIR"

capture_state() {
    local dir="$1"
    # Installed packages
    dpkg --get-selections > "$dir/packages.txt" 2>/dev/null || \
        rpm -qa --queryformat '%{NAME}\n' | sort > "$dir/packages.txt"

    # Running services
    systemctl list-units --type=service --state=running \
        --no-legend | awk '{print $1}' | sort > "$dir/services.txt"

    # Open ports
    ss -tlnup | awk 'NR>1 {print $5}' | sort -u > "$dir/ports.txt"

    # Critical file checksums
    sha256sum /etc/passwd /etc/sudoers /etc/ssh/sshd_config \
        $(find /etc/cron* -type f 2>/dev/null) > "$dir/checksums.txt"
}

if [ "$1" = "--baseline" ]; then
    capture_state "$BASELINE_DIR"
    echo "Baseline captured at $(date)"
    exit 0
fi

# Check mode
[ -d "$BASELINE_DIR" ] || { echo "No baseline. Run: $0 --baseline"; exit 1; }
capture_state "$CURRENT_DIR"

echo "=== DRIFT REPORT: $(date) ===" > "$REPORT"
for check in packages services ports checksums; do
    diff_out=$(diff "$BASELINE_DIR/$check.txt" "$CURRENT_DIR/$check.txt")
    if [ -n "$diff_out" ]; then
        echo "\n[DRIFT DETECTED] $check:" >> "$REPORT"
        echo "$diff_out" >> "$REPORT"
    fi
done

cat "$REPORT"
rm -rf "$CURRENT_DIR"
```

EXPLANATION

The baseline capture function creates a snapshot of system state: `dpkg --get-selections` or `rpm -qa` lists installed packages. `systemctl list-units --state=running` captures active services. `ss -tlnup` shows listening sockets. `sha256sum` on critical files detects tampering. The script has two modes: `--baseline` saves the reference state, normal run compares current state to baseline using `diff`. `diff` output shows + (new/added) and - (removed) lines. This detects: unauthorized package installations, new services starting, ports

opening, and critical config file modifications — all common indicators of compromise or misconfiguration drift.

PRO TIP

Integrate with a CI/CD pipeline to automatically compare production state against the desired state defined in your IaC repository.

REAL WORLD

Infrastructure drift detection bridges the gap between immutable infrastructure (ideal) and mutable real-world servers. It's the manual equivalent of what Puppet/Chef/Ansible do continuously.

40. Multi-Environment Deployment Script

SHELL SCRIPTING**ADVANCED**

#40

TASK

Write a deployment script that deploys an application to dev/staging/prod with: pre-flight checks, rollback capability, health verification, and Slack notifications.

SOLUTION

```
#!/bin/bash
ENV="${1:?Usage: $0 <dev|staging|prod> <version>}"
VERSION="${2:?}"
SLACK_WEBHOOK="${SLACK_WEBHOOK_URL}"

# Environment config
declare -A SERVERS
SERVERS[dev]="dev1.internal"
SERVERS[staging]="stg1.internal stg2.internal"
SERVERS[prod]="web1.prod web2.prod web3.prod"

[[ -v SERVERS[$ENV] ]] || { echo "Invalid environment: $ENV"; exit 1; }

notify() {
    local color="$1" msg="$2"
    curl -s -X POST "$SLACK_WEBHOOK" \
        -H "Content-type: application/json" \
        -d "{\"attachments\": [{ \"color\": \"$color\", \"text\": \"$msg\" } ]}" > /dev/null
}

rollback() {
    notify "danger" ":x: ROLLBACK triggered for $ENV v$VERSION"
    for server in ${SERVERS[$ENV]}; do
        ssh "$server" "systemctl restart app_previous || true"
    done
    exit 1
}

trap rollback ERR

notify "warning" ":rocket: Deploying v$VERSION to $ENV"

# Pre-flight checks
echo "Running pre-flight checks..."
[ "$(git tag -l "v$VERSION")" ] || { echo "Tag v$VERSION not found"; exit 1; }
```

```
curl -sf "http://artefact-repo/v$VERSION/app.tar.gz" -o /dev/null || { echo "Artifact
not found"; exit 1; }

for server in ${SERVERS[$ENV]}; do
  echo "Deploying to $server..."
  ssh "$server" "cd /opt/app && \
    cp -a . ../app_backup && \
    curl -s http://artefact-repo/v$VERSION/app.tar.gz | tar xz && \
    systemctl restart app && \
    sleep 5 && \
    curl -sf http://localhost:8080/health || exit 1"
done

notify "good" ":white_check_mark: v$VERSION deployed to $ENV successfully"
```

EXPLANATION

declare -A creates an associative array for environment-to-servers mapping. [[-v SERVERS[\$ENV]]] checks if the key exists (validates the environment argument). trap rollback ERR registers automatic rollback on any command failure — if any deployment step fails, rollback runs automatically. The notify function sends Slack messages with color coding (danger=red, warning=yellow, good=green). Pre-flight checks verify the git tag and artifact existence before touching servers. The deployment pipeline: backup current app, extract new version, restart service, wait 5 seconds, then verify the health endpoint. The health check failure triggers ERR which triggers rollback.

PRO TIP

In production, use blue-green or canary deployment patterns: deploy to half the servers, verify, then complete the rollout — never deploy to all instances simultaneously.

REAL WORLD

This pattern mirrors what tools like Capistrano, Fabric, and Ansible deploy roles implement. Understanding it helps debug and extend these tools.

09

Security & Hardening

5 assignments | Assignments 44 - 48

41. Comprehensive System Hardening Script

SECURITY & HARDENING

ADVANCED

#41

TASK

Write a CIS Benchmark-aligned hardening script covering: kernel parameters, filesystem hardening, network security, and account policies.

SOLUTION

```
#!/bin/bash
# CIS Benchmark Level 1 Hardening
echo "=== APPLYING SECURITY HARDENING ==="

# Kernel security parameters
cat > /etc/sysctl.d/99-security.conf << 'EOF'
# Disable IP source routing
net.ipv4.conf.all.accept_source_route = 0
net.ipv6.conf.all.accept_source_route = 0
# Disable ICMP redirects
net.ipv4.conf.all.accept_redirects = 0
net.ipv4.conf.all.secure_redirects = 0
# Enable SYN cookies (SYN flood protection)
net.ipv4.tcp_syncookies = 1
# Disable packet forwarding (not a router)
net.ipv4.ip_forward = 0
# Ignore ICMP broadcast pings
net.ipv4.icmp_echo_ignore_broadcasts = 1
# Log martian packets (impossible source addresses)
net.ipv4.conf.all.log_martians = 1
# Disable core dumps for SUID programs
fs.suid_dumpable = 0
# Enable ASLR (Address Space Layout Randomization)
kernel.randomize_va_space = 2
EOF
sysctl -p /etc/sysctl.d/99-security.conf

# Filesystem hardening - /etc/fstab additions
# Add: tmpfs /dev/shm tmpfs defaults,nodev,nosuid,noexec 0 0

# Account policy
sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS    90/' /etc/login.defs
sed -i 's/^PASS_MIN_DAYS.*/PASS_MIN_DAYS    7/' /etc/login.defs
sed -i 's/^PASS_WARN_AGE.*/PASS_WARN_AGE    14/' /etc/login.defs

# Lock system accounts
for user in $(awk -F: '$3<1000 && $1!="root" {print $1}' /etc/passwd); do
    usermod -L "$user" 2>/dev/null
done

echo "Hardening complete. Reboot recommended."
```

EXPLANATION

Source routing allows senders to specify the route their packets take — disabling it prevents IP spoofing attacks. ICMP redirects can be used to manipulate routing tables — disabling them prevents man-in-the-middle attacks. SYN cookies protect against SYN flood DoS attacks by encoding session information in the initial response rather than allocating kernel memory. Disabling IP forwarding prevents the server from acting as a router. ASLR (randomize_va_space=2) randomizes memory addresses, making buffer overflow exploits significantly harder. fs.suid_dumpable=0 prevents SUID programs from creating core dumps that could expose sensitive memory contents. Locking system accounts (UID<1000) prevents their use for interactive login.

PRO TIP

Run Lynis for a comprehensive automated security audit: `lynis audit system`. It checks hundreds of security items and provides a hardening index score.

REAL WORLD

CIS Benchmarks are the industry-standard security configuration guidelines used by cloud providers, enterprises, and compliance frameworks as baseline requirements.

42. Intrusion Detection with Tripwire/AIDE

SECURITY & HARDENING**ADVANCED**

#42

TASK

Set up AIDE (Advanced Intrusion Detection Environment) to baseline critical system files, run daily checks, and alert on changes to `/etc`, `/bin`, and `/usr/bin`.

SOLUTION

```
# Install AIDE
apt-get install -y aide aide-common

# Configure AIDE rules in /etc/aide/aide.conf
cat >> /etc/aide/aide.conf << 'EOF'
# Monitor critical directories
/etc      PERMS+md5+sha256
/bin      PERMS+md5+sha256
/sbin     PERMS+md5+sha256
/usr/bin  PERMS+md5+sha256
/usr/sbin PERMS+md5+sha256

# Exclude frequently changing files
!/etc/mtab
!/etc/aide/aide.db.new
!/var/log
EOF

# Initialize database (may take 10-20 minutes)
aide --init
mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Run check (compare current to baseline)
aide --check

# Update database after authorized changes
aide --update
mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Automate with cron
echo '0 3 * * * root aide --check | mail -s "AIDE Report $(hostname)"
admin@devopsshack.com' \
```

```
> /etc/cron.d/aide
```

EXPLANATION

AIDE creates a cryptographic database of file attributes: permissions, ownership, size, inode, and hash (MD5/SHA256). The PERMS rule monitors permission and ownership changes. md5+sha256 checks file content integrity. Lines starting with ! exclude paths from monitoring — log files and volatile files like mtab must be excluded to avoid false positives. aide --init scans the filesystem and creates the reference database. aide --check compares the current state to the database and reports any differences. After intentional changes (software updates, config changes), aide --update creates a new database that becomes the new baseline. The cron job runs nightly and emails results.

PRO TIP

Store the AIDE database on read-only media or a remote server — if an attacker can modify both the system and the local database, AIDE provides no protection.

REAL WORLD

AIDE is a requirement in PCI-DSS (Requirement 11.5) and NIST SP 800-53 for File Integrity Monitoring (FIM). It's the open-source equivalent of commercial tools like Tripwire Enterprise.

43. Fail2ban Custom Jail Configuration

SECURITY & HARDENING

INTERMEDIATE

#43

TASK

Configure fail2ban to protect: SSH (3 attempts/10min ban), Nginx 404 errors (10 per minute ban), and a custom application log. Create an email alert on each ban.

SOLUTION

```
# Install fail2ban
apt-get install -y fail2ban

# Create local config (never edit jail.conf directly)
cat > /etc/fail2ban/jail.local << 'EOF'
[DEFAULT]
bantime = 600
findtime = 600
maxretry = 5
action = %(action_mwl)s # Ban + email with whois + log lines
destemail = admin@devopsshack.com
sendername = Fail2Ban
mta = sendmail

[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
maxretry = 3

[nginx-404]
```

```

enabled = true
port = http,https
filter = nginx-404
logpath = /var/log/nginx/access.log
maxretry = 10
findtime = 60

[myapp]
enabled = true
port = 8080
filter = myapp-auth
logpath = /opt/myapp/logs/auth.log
maxretry = 5
EOF

# Create custom filter for nginx 404s
cat > /etc/fail2ban/filter.d/nginx-404.conf << 'EOF'
[Definition]
failregex = <HOST> .* "(GET|POST) .* HTTP/.*" 404
ignoreregex = .(jpg|png|css|js|ico|gif)$
EOF

systemctl enable --now fail2ban
fail2ban-client status

```

EXPLANATION

fail2ban monitors log files, detects patterns matching attack signatures (filters), and uses iptables/firewall to ban offending IPs. jail.conf defines 'jails' — each jail has a filter (regex pattern), log path, and action. Never edit jail.conf directly; create jail.local to override settings (survives package updates).bantime is how long IPs are banned (seconds). findtime is the observation window. maxretry is the allowed failures. action_mwl bans AND sends email with whois lookup and matching log lines. Custom filters use failregex with <HOST> as the IP placeholder. ignoreregex skips static asset 404s to reduce false positives. fail2ban-client status shows all jails and banned IPs.

PRO TIP

Use `fail2ban-client set jailname banip <IP>` for manual bans, and `unbanip` for manual unban. Check active bans with: `fail2ban-client status sshd`

REAL WORLD

Fail2ban is a first-line defense on any internet-facing server. Combined with rate limiting and geofencing, it significantly reduces brute-force attack surface.

44. GPG Key Management and File Signing

SECURITY & HARDENING

ADVANCED

#44

TASK

Generate a GPG key pair, sign a file to prove authenticity, verify signatures, encrypt a file for a recipient, and set up a keyserver workflow.

SOLUTION

```
# Generate key pair (batch mode)
gpg --batch --gen-key << 'EOF'
Key-Type: RSA
Key-Length: 4096
Subkey-Type: RSA
Subkey-Length: 4096
Name-Real: Aditya Jaiswal
Name-Email: admin@devopsshack.com
Expire-Date: 1y
%no-protection
%commit
EOF

# List keys
gpg --list-keys
gpg --list-secret-keys

# Sign a file (detached signature)
gpg --armor --detach-sign release-v2.0.tar.gz
# Creates: release-v2.0.tar.gz.asc

# Verify signature
gpg --verify release-v2.0.tar.gz.asc release-v2.0.tar.gz

# Encrypt for recipient (they must have your public key)
gpg --armor --recipient admin@devopsshack.com --encrypt secrets.txt

# Export public key (share with others)
gpg --armor --export admin@devopsshack.com > devopsshack-pubkey.asc

# Import public key
gpg --import colleague-pubkey.asc

# Upload to keyserver
gpg --keyserver keys.openpgp.org --send-keys <KEY_ID>
```

EXPLANATION

GPG (GNU Privacy Guard) implements the OpenPGP standard for encryption and signing. A key pair consists of a public key (share freely) and private key (keep secret). Detached signatures (`--detach-sign`) create a separate `.asc` file — the original is untouched, anyone with your public key can verify authenticity. `--armor` outputs base64-encoded text suitable for email/files. Encryption with `--encrypt` uses the recipient's public key — only their private key can decrypt. Key signing (`--sign-key`) establishes trust chains. 4096-bit RSA provides strong security. 1y expiry forces key renewal and limits exposure if compromised. Keyservers distribute public keys for discovery.

PRO TIP

Use `gpg --card-status` for hardware security keys (YubiKey). Private keys stored on hardware tokens cannot be exported, providing strongest protection.

REAL WORLD

GPG signing is used for software release authentication (apt/yum repository signing), secure configuration distribution, and compliance requirements for code signing.

45. AppArmor Profile Creation

SECURITY & HARDENING

ADVANCED

#45

TASK

Create an AppArmor profile for nginx that restricts it to only the files and capabilities it legitimately needs, and test the profile in complain mode before enforcing.

SOLUTION

```
# Install AppArmor utilities
apt-get install -y apparmor-utils

# Generate initial profile by running nginx under aa-genprof
systemctl stop nginx
aa-genprof /usr/sbin/nginx

# In another terminal: systemctl start nginx && curl http://localhost
# Back in aa-genprof: press S to scan, A to allow found accesses, F to finish

# View generated profile
cat /etc/apparmor.d/usr.sbin.nginx

# Manual profile example:
cat > /etc/apparmor.d/usr.sbin.nginx << 'EOF'
#include <tunables/global>
/usr/sbin/nginx {
    #include <abstractions/base>
    #include <abstractions/namespace>

    capability net_bind_service, # Bind to ports <1024
    capability setuid,
    capability setgid,

    /usr/sbin/nginx mr,
    /etc/nginx/** r,
    /var/www/html/** r,
    /var/log/nginx/*.log w,
    /run/nginx.pid rw,
    /tmp/ rw,
    /tmp/** rw,

    deny /etc/shadow r,
    deny /etc/passwd w,
}
EOF

# Test in complain mode first (logs but doesn't block)
aa-complain /usr/sbin/nginx
aa-logprof # Review and update from syslog

# Enforce when satisfied
aa-enforce /usr/sbin/nginx
apparmor_status
```

EXPLANATION

AppArmor (Application Armor) is a mandatory access control (MAC) system built into the Linux kernel. Unlike DAC (file permissions), MAC rules are enforced regardless of user identity — even root cannot bypass them. Profiles define exactly what files a program can access (r=read, w=write, m=mmap, x=execute) and what capabilities it can use. `capability net_bind_service` allows binding to privileged ports (<1024) without running as root. `deny` rules explicitly block access. `aa-genprof` runs the application and learns its access patterns (like a MAC learning switch). `Complain` mode logs violations without blocking — essential for testing before enforcing. `aa-logprof` reads the log and interactively refines the profile.

PRO TIP

SELinux is the alternative MAC system used in RHEL/CentOS — more powerful but harder to configure. Use `audit2allow` to generate policies from `audit.log` denials.

REAL WORLD

AppArmor profiles limit the blast radius if a service is compromised — even a root exploit in nginx cannot escape its sandbox to read `/etc/shadow` or write outside `/var/www`.

10

Package Management

46. Custom APT/YUM Repository Setup

PACKAGE MANAGEMENT

ADVANCED

#46

TASK

Create a local APT repository from downloaded .deb packages, sign it with GPG, configure it on client machines, and keep it synchronized.

SOLUTION

```
# Install tools
apt-get install -y dpkg-dev apt-utils gnupg

# Create repository directory structure
mkdir -p /var/www/html/repo/dists/stable/main/binary-amd64
mkdir -p /var/www/html/repo/pool/main

# Copy packages
cp *.deb /var/www/html/repo/pool/main/

# Generate Packages index
cd /var/www/html/repo
dpkg-scanpackages pool/main | gzip > dists/stable/main/binary-amd64/Packages.gz
dpkg-scanpackages pool/main > dists/stable/main/binary-amd64/Packages

# Create Release file
cat > dists/stable/Release << 'EOF'
Archive: stable
Origin: DevOps Shack Internal
Label: DevOps Shack
Architecture: amd64
Component: main
EOF

apt-ftparchive release dists/stable >> dists/stable/Release

# Sign with GPG
gpg --armor --detach-sign -o dists/stable/Release.gpg dists/stable/Release
gpg --armor --clearsign -o dists/stable/InRelease dists/stable/Release

# On clients - add repository:
wget -qO- http://repo-server/repo/pubkey.gpg | apt-key add -
echo 'deb http://repo-server/repo stable main' > /etc/apt/sources.list.d/internal.list
apt-get update
```

EXPLANATION

An APT repository requires: pool/ directory (actual .deb files), dists/ directory (metadata indexes), and a Release file with checksums. dpkg-scanpackages scans the pool and generates a Packages file listing all available packages with their metadata and checksums. gzip compresses for faster downloading. The Release file describes the repository and contains SHA256 checksums of all index files — this creates a chain of trust. GPG signing: Release.gpg is a detached signature, InRelease is a cleartext signed version (newer clients prefer InRelease). Clients import the public key to verify signatures. This prevents package tampering in transit.

PRO TIP

For a simpler setup, use `reprepro` for Debian or `createrepo_c` for RPM repositories — they automate much of the metadata generation and signing.

REAL WORLD

Internal package repositories are essential for air-gapped environments, ensuring all servers use approved, tested package versions, and reducing internet dependency.

47. Snap and Flatpak Management

PACKAGE MANAGEMENT**INTERMEDIATE**

#47

TASK

Compare `snap` vs `flatpak` for application isolation, install the same app from both, investigate their sandbox configurations, and manage their permissions.

SOLUTION

```
# SNAP
# Install snapd
apt-get install -y snapd

# Install an app
snap install vlc

# List installed snaps
snap list

# Check snap interfaces (permissions)
snap interfaces vlc

# Connect/disconnect interface (e.g., camera access)
snap connect vlc:camera
snap disconnect vlc:camera

# Check snap confinement level
snap info vlc | grep confinement

# Run in strict mode vs devmode
snap run vlc

# FLATPAK
# Install flatpak
apt-get install -y flatpak
flatpak remote-add --if-not-exists flathub https://flathub.org/repo/flathub.flatpakrepo

# Install app
flatpak install flathub org.videolan.VLC

# List installed
flatpak list

# Check permissions
flatpak info --show-permissions org.videolan.VLC

# Override permissions
flatpak override --user --nodevice=dri org.videolan.VLC
flatpak override --user --filesystem=home:ro org.videolan.VLC
```

EXPLANATION

Snap (Canonical) and Flatpak (Red Hat/GNOME) are universal package formats with sandboxing. Snap packages include all dependencies and run in strict confinement using AppArmor (snaps) or bubblewrap+seccomp (flatpak). Snap interfaces are predefined connection points: connecting vlc:camera grants camera access; disconnecting revokes it. Confinement levels: strict (fully sandboxed), classic (like traditional packages, no sandbox), devmode (developer mode, all access). Flatpak uses portals for controlled access to host resources. flatpak override allows per-user, per-app permission customization: --nodevice=dri disables GPU access, --filesystem=home:ro grants read-only home directory access. Both distribute through their own stores (Snap Store, Flathub).

PRO TIP

For server applications, prefer traditional .deb/.rpm packages — snaps and flatpaks add overhead and complexity not needed in server environments.

REAL WORLD

Understanding these formats is important for desktop Linux deployments and IoT/edge devices where application isolation is critical for security.

48. Dependency Resolution and Conflict Management

PACKAGE MANAGEMENT**ADVANCED**

#48

TASK

Diagnose and resolve a complex package dependency conflict where two packages require incompatible versions of a shared library. Use apt-cache, dpkg, and manual pinning.

SOLUTION

```
# Diagnose the conflict
apt-cache show conflicting-package | grep Depends
apt-cache policy conflicting-package

# Find what provides a specific library version
apt-cache showpkg libssl1.1
apt-rdepends libssl1.1 # Reverse dependencies

# Check currently installed version
dpkg -l libssl*

# Simulate apt install to see conflict
apt-get install -s target-package 2>&1 | grep -E 'conflict|error|cannot'

# Resolution Option 1: Pin a specific version
cat > /etc/apt/preferences.d/libssl << 'EOF'
Package: libssl1.1
Pin: version 1.1.1n-0+deb11u5
Pin-Priority: 1001
EOF

# Resolution Option 2: Install from specific release
apt-get install -t bullseye libssl1.1

# Resolution Option 3: Install .deb directly
dpkg -i libssl1.1_1.1.1n_amd64.deb
apt-get install -f # Fix any remaining deps

# Verify resolution
```

```
apt-cache policy libssl1.1
dpkg --verify target-package
# Check for broken packages
dpkg --audit
apt-get check
```

EXPLANATION

apt-cache show displays package metadata including dependencies. apt-cache policy shows available versions and priorities from each repository. apt-rdepends shows what packages depend on a library (helps assess impact of version changes). -s flag in apt-get install simulates without modifying the system. APT pinning (/etc/apt/preferences.d/) overrides version selection: Pin-Priority 1001 installs even if it would downgrade. -t flag installs from a specific suite (codename). dpkg -i installs a .deb directly, bypassing apt resolution — apt-get install -f then resolves any resulting unmet dependencies. dpkg --audit detects installed packages with missing dependencies. apt-get check verifies the dependency consistency of the package database.

PRO TIP

For Python/Ruby/Node.js version conflicts, use pyenv, rbenv, or nvm instead of system packages — they allow multiple versions to coexist without conflicts.

REAL WORLD

Dependency hell is a real operational challenge, especially on long-running servers mixing packages from different sources. Understanding resolution strategies prevents upgrade emergencies.

49. Container Image Layer Analysis

PACKAGE MANAGEMENT

ADVANCED

#49

TASK

Analyze a Docker image for size optimization opportunities, identify unnecessary packages, find security vulnerabilities, and rebuild with a minimal footprint.

SOLUTION

```
# Inspect image layers and sizes
docker image history nginx:latest
docker inspect nginx:latest | jq '[0].RootFS.Layers | length'

# Detailed layer analysis with dive
dive nginx:latest

# Scan for vulnerabilities
trivy image nginx:latest
docker scout cves nginx:latest

# Export and analyze filesystem
docker export $(docker create nginx:latest) | \
  tar -t | grep -E '\.(deb|rpm)$'

# Find largest directories in image
```

```
docker run --rm nginx:latest du -sh /* 2>/dev/null | sort -rh | head -20

# Optimized multi-stage Dockerfile example:
# FROM golang:1.21 AS builder
# WORKDIR /app
# COPY . .
# RUN CGO_ENABLED=0 GOOS=linux go build -ldflags='-w -s' -o server
#
# FROM gcr.io/distroless/static:nonroot
# COPY --from=builder /app/server /server
# USER nonroot:nonroot
# ENTRYPOINT ["/server"]

docker build -t myapp:optimized .
docker images myapp:optimized # Compare sizes
```

EXPLANATION

docker image history shows each layer, its command, and size. Each RUN, COPY, ADD instruction creates a new layer. dive is an interactive tool that shows layer contents and highlights wasted space. trivy and docker scout scan images against vulnerability databases (CVE databases). Multi-stage builds are the primary optimization technique: a large build stage compiles the application, then only the compiled binary is copied to a minimal runtime image. CGO_ENABLED=0 creates a statically linked binary with no C library dependencies. -ldflags='-w -s' strips debug symbols. gcr.io/distroless contains only the runtime (no shell, no package manager, no utilities) — radically reducing attack surface and image size.

PRO TIP

Target image sizes: Go/Rust apps with distroless: <20MB. Python apps with slim base: <100MB. Java apps with JRE: <200MB. Anything larger deserves investigation.

REAL WORLD

Container image optimization directly impacts: pull time (deployment speed), registry storage costs, and attack surface (smaller = fewer vulnerabilities).

11

Performance Tuning

4 assignments | Assignments 50-59

50. Complete Production Server Tuning

PERFORMANCE TUNING

ADVANCED

#50

TASK

Apply a comprehensive performance tuning profile for a high-traffic web server: tune the kernel TCP stack, optimize disk I/O schedulers, configure huge pages, and profile with perf.

SOLUTION

```
# ——— KERNEL TCP TUNING ———
cat > /etc/sysctl.d/99-perf.conf << 'EOF'
# Increase socket buffer sizes
net.core.rmem_max = 134217728
net.core.wmem_max = 134217728
net.ipv4.tcp_rmem = 4096 87380 67108864
net.ipv4.tcp_wmem = 4096 65536 67108864
# Enable TCP Fast Open
net.ipv4.tcp_fastopen = 3
# Reduce TIME_WAIT impact
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp_fin_timeout = 15
# Increase connection backlog
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 65535
# Increase local port range
net.ipv4.ip_local_port_range = 1024 65535
# TCP keepalive tuning
net.ipv4.tcp_keepalive_time = 300
net.ipv4.tcp_keepalive_probes = 5
net.ipv4.tcp_keepalive_intvl = 15
EOF
sysctl -p /etc/sysctl.d/99-perf.conf

# ——— DISK I/O SCHEDULER ———
# For SSDs: use 'none' or 'mq-deadline'
for disk in /sys/block/sd*/queue/scheduler; do
    echo 'mq-deadline' > "$disk"
done
# For NVMe: already uses 'none'

# ——— HUGE PAGES ———
# Configure 2MB huge pages for databases (e.g., 2GB total)
echo 1024 > /proc/sys/vm/nr_hugepages
echo 'vm.nr_hugepages = 1024' >> /etc/sysctl.d/99-perf.conf

# ——— PROFILING WITH PERF ———
# Install
apt-get install -y linux-tools-$(uname -r)

# CPU profiling: find hotspots in nginx
perf top -p $(pgrep nginx | head -1)

# Record 30-second profile
perf record -g -p $(pgrep nginx | head -1) sleep 30
perf report --stdio | head -50

# Flame graph generation
```

```
perf record -g ./myapp &
perf script | stackcollapse-perf.pl | flamegraph.pl > flamegraph.svg

# — VERIFY —
sysctl net.core.somaxconn
cat /sys/block/sda/queue/scheduler
grep HugePages /proc/meminfo
```

EXPLANATION

TCP buffer tuning: rmem/wmem are minimum, default, maximum receive/send buffer sizes. Larger buffers improve throughput for high-latency connections (BDP = bandwidth * latency). TCP Fast Open (TFO=3 enables both client and server) eliminates one RTT for repeat connections by sending data in the SYN packet. tcp_tw_reuse allows reusing TIME_WAIT sockets for new connections, solving port exhaustion under high connection rates. somaxconn and tcp_max_syn_backlog control connection queue depth — critical for traffic spikes. I/O schedulers: CFQ/BFQ are designed for spinning disks with physical seek optimization; SSDs/NVMe benefit from 'none' (no reordering needed) or 'mq-deadline' (prevents starvation). Huge pages (2MB vs 4KB) reduce TLB (Translation Lookaside Buffer) pressure for memory-intensive applications like databases. perf record captures CPU samples with call stack; perf report shows where time is spent. Flame graphs visualize the full call stack hierarchy as a proportional chart.

PRO TIP

Use pprof for Go applications, async-profiler for JVM, and py-spy for Python. Always profile before tuning — optimization without measurement is guesswork.

REAL WORLD

These tuning parameters can 2-3x the throughput of a web server under high load. They are applied by default in high-performance Linux distributions tuned for server workloads.

Keep Learning. Keep Building.

Aditya Jaiswal | @devopsshack | devopsshack.com